

JSF – JAVASERVER PAGES II

JSF: Facelets

Definição e Contexto:

- Facelet é a tecnologia de view padrão para JSF, substituindo o JSP. Oferece um sistema de templates baseado em XML para a construção de interfaces de usuário em aplicações JSF.

Vantagens sobre JSP:

- **Performance:** melhor performance por evitar a fase de compilação do JSP para servlets.
 - O JSP continha códigos Java dinâmicos, então o código tinha que ser compilado para assim se tornar um HTML estático. Isso é um problema de performance quando comparado ao JSF.
- **Reuso de Código:** facilita a reutilização de componentes e a criação de templates, permitindo uma abordagem mais modular.
- **Expression Language Aprimorada:** suporta o uso extensivo da Expression Language (EL) para vinculação de dados e integração de lógica de backend.



5m

```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml"
3     xmlns:h="http://xmlns.jcp.org/jsf/html"
4     xmlns:f="http://xmlns.jcp.org/jsf/core">
5 <h:head> <title>Cadastro de Usuário</title> </h:head>
6 <h:body>
7     <h:form>
8         <h:panelGrid columns="3">
9             <!-- InputText para Username -->
10            <h:outputLabel for="username" value="Username:" />
11            <h:inputText id="username" value="#{userBean.username}" required="true"
12                requiredMessage="O username é obrigatório." />
13
14            <!-- InputSecret para Password -->
15            <h:outputLabel for="password" value="Password:" />
16            <h:inputSecret id="password" value="#{userBean.password}" required="true"
17                reDisplay="false" requiredMessage="A senha é obrigatória." />
18        </h:panelGrid>

```

```
19      <!-- SelectOneMenu para Versões do Java -->
20      <h:outputLabel for="javaVersion" value="Versão do Java:"/>
21      <h:selectOneMenu id="javaVersion" value="#{userBean.javaVersion}">
22          <f:selectItem itemValue="8" itemLabel="Java 8"/>
23          <f:selectItem itemValue="9" itemLabel="Java 9"/>
24      </h:selectOneMenu>
25
26      <!-- SelectOneRadio para RadioButton -->
27      <h:outputLabel value="Preferência de Contato:"/>
28      <h:selectOneRadio id="contactPreference" value="#{userBean.contactPreference}">
29          <f:selectItem itemValue="email" itemLabel="Email"/>
30          <f:selectItem itemValue="phone" itemLabel="Telefone"/>
31      </h:selectOneRadio>
32
33      <!-- CommandButton para Submissão -->
34      <h:commandButton value="Registrar" action="#{userBean.register}"/>
35  </h:panelGrid>
36  </h:form>    <h:messages/>
37 </h:body>
38 </html>
```

```
1 @ManagedBean
2 @RequestScoped
3 public class UserBean {
4     private String username;
5     private String password;
6     private String javaVersion;
7     private String contactPreference;
8
9     // Getters e Setters omitidos
10
11     // Método chamado ao submeter o formulário
12     public String register() {
13         Severity severidade = FacesMessage.SEVERITY_INFO;
14         String resumo = "Registro bem-sucedido";
15         String detalhe = "Usuário " + username + " registrado com sucesso!";
16
17         //criando uma mensagem global
18         FacesMessage msg = new FacesMessage(severidade, resumo, detalhe);
19         FacesContext.getCurrentInstance().addMessage(null, msg);
20
21         // Retorna null para permanecer na mesma página
22         return null;
23     }
24 }
```



O código visto anteriormente gera este resultado:

- Registro bem-sucedido

O campo de ID fica null para que possa ser preenchido, se não, deixa de ser uma mensagem global.

Deve-se usar o Input Text e o Input Secret.

DIRETO DO CONCURSO

7. (FCC/TRT/23/2016) Em um site que utiliza JSF, um Analista está criando um pequeno formulário para entrada do nome do usuário e da senha de acesso a uma área restrita. O nome do usuário poderá ficar legível, porém, a senha não. Os elementos JSF que deverão ser utilizados para criar o campo nome do usuário e senha são, respectivamente,

- a. < h:inputText/> e < h:inputSecret/>
- b. < form:inputText/> e < form:inputPassword/>
- c. < html:inputField/> e < html:inputHidden/>
- d. < h:inputTextarea/> e < h:inputPassword/>
- e. < h:inputField/> e < h:inputHidden/>



Deve-se usar o Input Text e o Input Secret.

8. (FCC/TRF–3/2016) Um Técnico Judiciário de TI está criando o formulário de um site para submeter imagens utilizando JSF 2.2. Nesse formulário, para criar o campo para upload da imagem e para criar um botão para submeter a imagem utilizam-se, respectivamente, as tags

- a. < file:inputFile> e <html:submitButton>
- b. < h:uploadImage> e <h:uploadButton>
- c. < f:uploadFile> e <h:submitButton>
- d. < jsf:inputFile> e <jsf:uploadButton>
- e. < h:inputFile> e <h:commandButton>



Ele quer criar um botão para submeter o formulário, então, para isso, deve-se usar um “command button”.

9. (AOCP/CASAN/2016) Assinale a alternativa que apresenta a linha que possui um erro.

- 1. <h:form>
- 2. <h:panelGrid columns="2" rendered="#{!login.loggedIn}">
- 3. <h:outputLabel for="username">Username:</h:outputLabel>
- 4. <h:outputLabel id="username" value="#{credentials.username}"/>
- 5. <h:outputLabel for="password">Password:</h:outputLabel>
- 6. <h:inputText id="password" value="#{credentials.password}"/>
- 7. </h:panelGrid>
- 8. <h:commandButton value="Login" action="#{login.login}" rendered="#{!login.loggedIn}"/>
- 9. <h:commandButton value="Logout" acion="#{login.logout}" rendered="#{login.loggedIn}"/>
- 10. </h:form>

- a. 3
- b. 4
- c. 5
- d. 6
- e. 7



A linha 2 só acontece se a pessoa não estiver logada.

Na questão, há erro, pois tem um output label seguido diretamente por outro output label, quando deveria ser um output label seguido por um input text.



10. (FCC/TRF-3/2016) Em uma aplicação que utiliza JSF, uma mensagem global foi definida por meio da instrução abaixo: **FacesMessage m = new FacesMessage("Dados do funcionário do TRESP salvos com sucesso!");** Para adicionar essa mensagem ao contexto do JSF utiliza-se a instrução

- a. FacesContext.getInstance().addMessage("formFuncionario:idFun", m);
- b. PrimeFaces.getInstance().addMessage(frmFunc:idFun, m);
- c. m.getContext().ddMessage();
- d. FacesContext.getCurrentInstance().addMessage(null, m);
- e. PrimeFaces.setContext(m);



Para adicionar essa mensagem, usa-se a instrução `acesContext.getInstance().addMessage("formFuncionario:idFun", m).`

11. (FCC/TRF-3/2019) Considere o trecho abaixo em uma página web que utiliza JSF.

```
<h:form>
  <h:inputText value="....." I />
</h:form>
```

Considere que na mesma aplicação, em condições ideais, exista a classe Java abaixo.

```
public class Pessoa {
    private String nome;
    public void serNome(String nome) {
        this.nome=nome;
    }
    public String getNome() {
        return this.nome;
    }
}
```

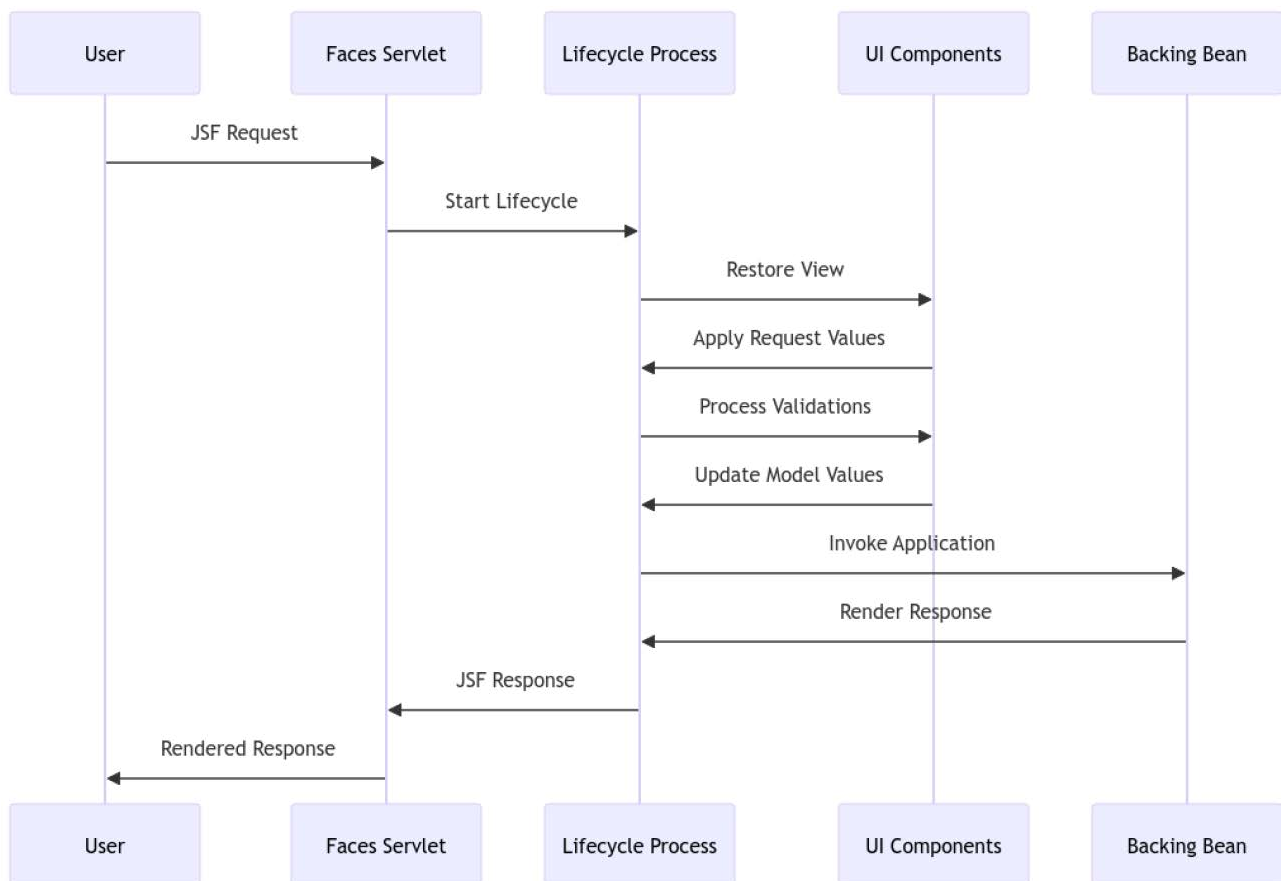
Para que seja obtida uma instância de Pessoa e chamado o método getNome a partir dessa instância,mostrando no campo do formulário o nome obtido, a lacuna i deve ser preenchida corretamente por:

- a. `pessoa.getNome()`
- b. `{ pessoa.getNome();}`
- c. `jsf->pessoa.nome`
- d. `# [pessoa.getNome()]`
- e. `# {pessoa.nome}`



`# {pessoa.nome}`

JSF: Ciclo de Vida



1. Restore View (R de restaurar): a fase inicial onde o JSF constrói ou restaura a árvore de componentes da UI para a página solicitada. Se é a primeira vez que a página é acessada, uma nova árvore é construída. Se é um postback (uma submissão de formulário), a árvore existente é restaurada.

2. Apply Request Values (A de aplicar): cada componente da UI na árvore de componentes processa os dados recebidos da requisição, convertendo-os e validando-os conforme necessário.

3. Process Validations(P de processar): Nesta fase, são executadas as validações nos dados de entrada. Se algum dado não passar na validação, a fase de resposta é chamada diretamente, e o processo de atualização do modelo e invocação de aplicativo é ignorado.

- Se algum dado não é aprovado na validação, a fase de resposta é chamada diretamente.

4. Update Model Values(A de atualizar): os valores validados são transferidos para as propriedades dos beans correspondentes, atualizando o modelo de dados da aplicação.

5. Invoke Application(I de invocar): o JSF invoca a lógica de aplicativo, como a chamada de métodos de ação em managed beans, que podem resultar em navegação para outra página.

6. Render Response(R de renderizar): na última fase, o JSF gera a resposta HTML a ser enviada de volta ao cliente, renderizando a árvore de componentes da UI para HTML.



RAPIAR

Restaura, Aplica, Processa, Atualiza, Invoca, Renderiza.

DIRETO DO CONCURSO

12. (CEBRASPE/TJ/PA/2020) Assinale a opção que apresenta a fase do ciclo de vida que, no framework JavaServer Faces 2.0, recria a árvore de componentes da página, dispara validadores nos componentes da UIViewRoot e a atualiza dentro do objeto FacesContext.

- Apply Request Values
- Restore View
- Process Validations
- Invoke Application
- Render Response



Árvore é a primeira fase, a restore view.

13. (IADES/TRE-PA/2014) O ciclo de vida do JavaServer Faces (JSF) é composto por seis fases. Acerca desse tema, assinale a alternativa correta.

- a. A fase em que se atualizam valores do modelo só é alcançada após todos os componentes JSF serem considerados válidos na fase que processa as validações.
- b. Na fase em que se aplicam os valores de requisição, cada componente extrai seu novo valor e armazena em si próprio; porém, caso esse valor não seja do tipo String, uma mensagem de erro é gerada e associada ao componente.
- c. Quando é feita uma requisição a uma página JSF, sua implementação inicia a fase que processa as validações dos componentes.
- d. Durante a fase de atualizar os valores do modelo, a implementação do JSF manipula eventos da aplicação, como ir para a próxima página por meio de um link.
- e. Caso sejam encontrados erros nas fases em que as validações são processadas ou em que os valores do modelo são atualizados, tais erros são exibidos na página após a fase de renderizar a resposta, independentemente de a página conter um ou mais componentes `<message/>` ou `<messages/>`.



A fase em que se atualizam valores do modelo só é alcançada após todos os componentes JSF serem considerados válidos na fase que processa as validações.

JSF: Scopes

Escopos JSF 2.2 ou Inferior:

@RequestScoped: duração da requisição HTTP.

@SessionScoped: duração da sessão HTTP do usuário.

@ApplicationScoped: compartilhado durante a vida útil da aplicação web.

@ViewScoped: persiste durante a visualização de uma página JSF.

@NoneScoped: bean não gerenciado, sem ciclo de vida.

@CustomScoped: permite definir um ciclo de vida personalizado.

Escopos CDI:

@RequestScoped: Similar ao JSF, com integração em várias tecnologias.

@SessionScoped: Similar ao JSF, duração da sessão HTTP.

@ApplicationScoped: Similar ao JSF, compartilhado por toda a aplicação.

@Dependent: O escopo padrão, a duração depende do bean consumidor.

@ConversationScoped: Mantém o estado em uma série de interações.

@Transactional: Gerenciamento do bean dentro de uma transação JTA.

DIRETO DO CONCURSO

14. (FGV/PROCEMPA/2014) Para uma aplicação JSF usar um bean que injeta outra classe bean, o primeiro bean precisa ser capaz de manter seu estado por um certo tempo, que será especificado em sua anotação de escopo. Acerca da definição de escopos em aplicações JSF, correlacione as duas colunas a seguir:

- 1) @RequestScoped
- 2) @SessionScoped
- 3) @ApplicationScoped

- () o estado do bean persiste por múltiplas requisições HTTP provenientes de um mesmo usuário.
- () o estado do bean persiste por todo o tempo de vida da aplicação WEB.
- () o estado do bean persiste apenas por uma única requisição HTTP.

- a. 3 – 1 – 2
- b. 3 – 2 – 1
- c. 1 – 2 – 3
- d. 2 – 3 – 1
- e. 2 – 1 – 3



@SessionScoped: o estado do bean persiste por múltiplas requisições HTTP provenientes de um mesmo usuário.

@ApplicationScoped: o estado do bean persiste por todo o tempo de vida da aplicação WEB.

@RequestScoped: o estado do bean persiste apenas por uma única requisição HTTP.

GABARITO

- 7. a
- 8. e
- 9. b
- 10. d
- 11. d
- 12. b
- 13. a
- 14. d

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pelo professor Gunter Ribeiro Amorim.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
